

DESIGNING A 32-BYTE RAM

Note:

- Use Ubuntu OS since GTKWave is available in Ubuntu
 - Upload your solution on Quanta by 5.50 pm.
-

This lab aims to simulate a 32-byte RAM. This RAM uses D-flip-flops to store each bit.

Each 8 bits are packed together to make a byte called registers. Although a register contains 8 bits only, the user wants to access 16 bits together. Hence, we will use the Big-Endian format to store the data (the lower address will have the higher byte and vice versa).

Since the user accesses 16 bits (2 bytes as a whole), there will be a total of $32/2 = 16$ memory locations and hence 16 addresses. To access these 16 addresses, we will need 4 bits to identify the address ($2^4 = 16$). Hence, we will use a decoder (4 to 16) that will select each memory location.

Finally, for the output, we will have to use a mux that selects 2 bytes out of the 32 bytes, depending upon the address given by the user.

Some important points to note:

1. Since this lab will be auto-evaluated, make sure you follow all implementation details properly and write runnable code.
2. Each module may have partial marks, but since some modules depend on others, follow the order given below.
3. The modules are already declared in their respective files and you only need to complete the logic of the body. Make sure you don't modify the header.
4. You need to open the **impl** folder in VSCode.
5. You can use **python3 ram_test.py** to check your code for correctness after completing each module.
6. After finishing, use **python3 create_submission.py** to create the submission zip and submit it on Quanta. Only submissions that follow this will be evaluated.
7. If you get a permission denied error, use **chmod +x './DO NOT OPEN/dist/ram_test_final'**
8. The output folder contains the expected outputs for each module.

The implementation of this is divided into several modules (Please follow the given order, as the modules are dependent on each other)

1. `D_ff`(clk, chipSelect, reset, regWrite, enable, d, q)

- a. The D flip flop must work at the positive edge of the clock.
- b. Since the user can have multiple such chips, he gives us a chipSelect input which signifies if this chip is selected.

- c. If the chip is selected and the reset input is high, the output q should be 0. (This should also happen only at the posedge of the clock).
- d. If the chip is selected, the D flip flop is enabled, and regWrite is high, the input d is written into the output q.
- e. Please be aware that there is no need to make modifications to the D_ff module. It has been provided to you in its entirety to prevent any fundamental errors.

2. mux2to1_16bits(in0, in1, select, muxOut) (1 mark)

- a. This is a 16 bit mux that selects in0 or in1 as muxOut, depending upon the select input.
- b. This would be used in the memory module when outputEnable would be 0. More on that later. Here, you need to define the mux as a normal 16-bit 2-to-1 mux.

3. decoder4to16(destReg, decOut) (2 mark)

- a. This decoder is used to decode the 4 bit address destReg and hence selects one out of the 16 possible memory locations.

4. register8bit(clk, chipSelect, reset, regWrite, enable, inR, outR) (2 mark)

- a. This represents a byte in the memory. Each register is made up of 8 D flip flops.
- b. inR is the 8 bit out of the 16 bit data that the user may want to write.
- c. Similarly, outR is 8 bits that the user would read.

5. mux32to2(in0, in1, in2, ... in31, select, muxOut) (2 marks)

- a. This is a 32-BYTE to 2-BYTE mux that selects 2 bytes out of the 32 bytes, based on the select input.
- b. This would be used for reading the memory.
- c. Wherein, the ins would be the 32 bytes of the memory
- d. select would be the 4 bit address given by the user(the memory location that the user wants to access)
- e. muxOut would be the 16 bits of the memory location.
- f. Please note that the memory follows the Big-Endian format. i.e. the most significant byte of a multi-byte data type is stored at the lowest memory address, and the least significant byte is stored at the highest memory address. It is as if the data is read from left to right.

E.g. Data [15:0]

Memory location 0 - Data[15:8]

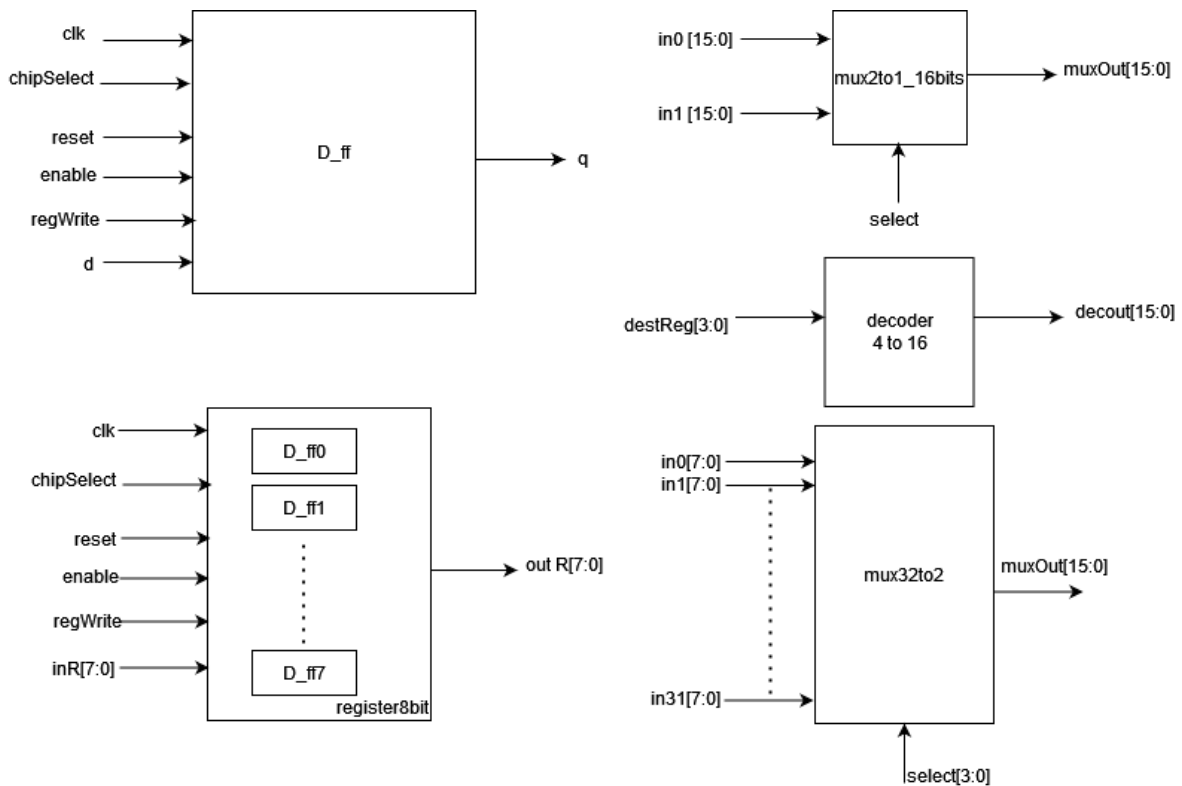
Memory location 1 - Data[7:0]

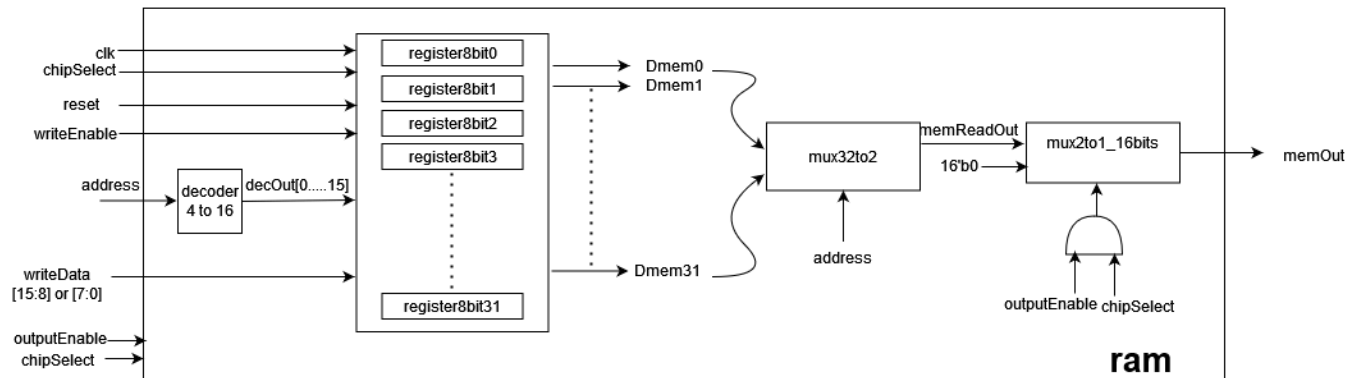
6. ram(clk, chipSelect, reset, outputEnable, writeEnable, address, writeData, memOut) (3 marks)

- a. The whole ram works at the positive edge of the clock, that too only if the chip is selected.

- b. The ram must only be written at the positive edge of the clock, but can be read anytime(only if the chip is selected).
- c. The memory has 16 memory locations, and each location has 2 bytes. Hence total of 32 bytes.
- d. Reset is used to reset the memory i.e. if the chip is selected and reset is high, all the bits in the memory become 0.
- e. outputEnable would be high if the user wants to read the memory, in which case memOut should be the 16 bits stored in the memory location pointed to by the 4-bit address
- f. writeEnable would be high if the user wants to write into the memory, in which case writeData would be the 16 bits that the user wants to write into the memory location pointed to by the 4-bit address.

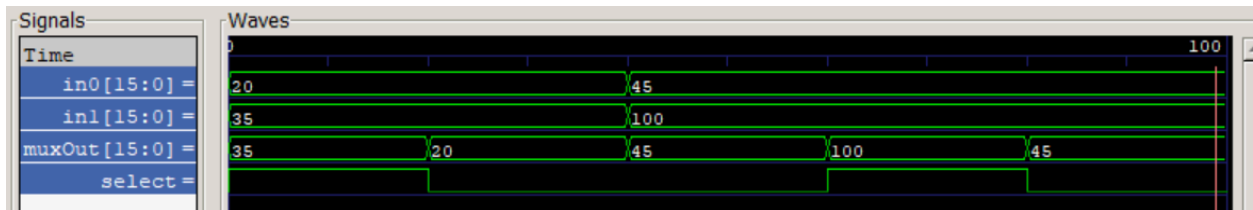
Circuit Diagram for each module



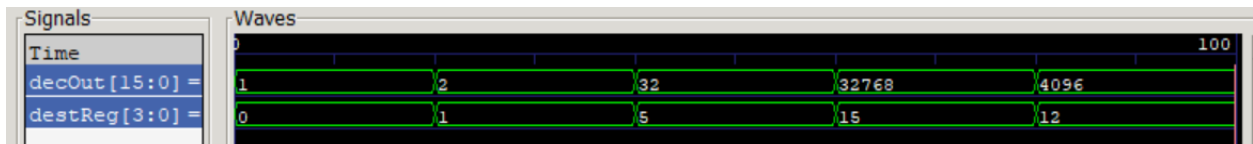


Waveform Generated

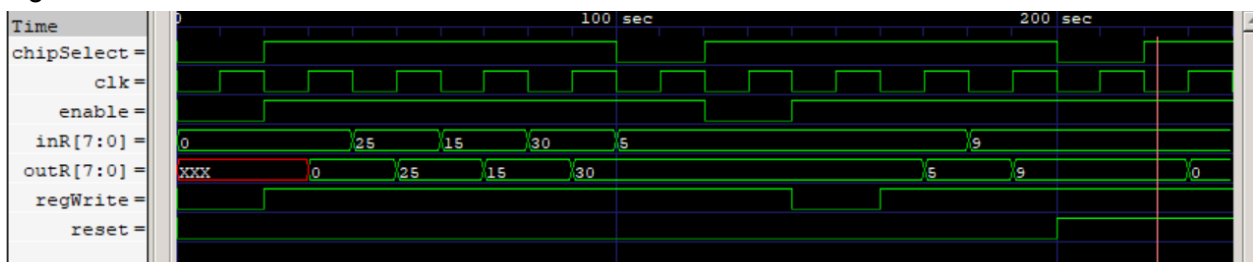
mux2to1_16bits:



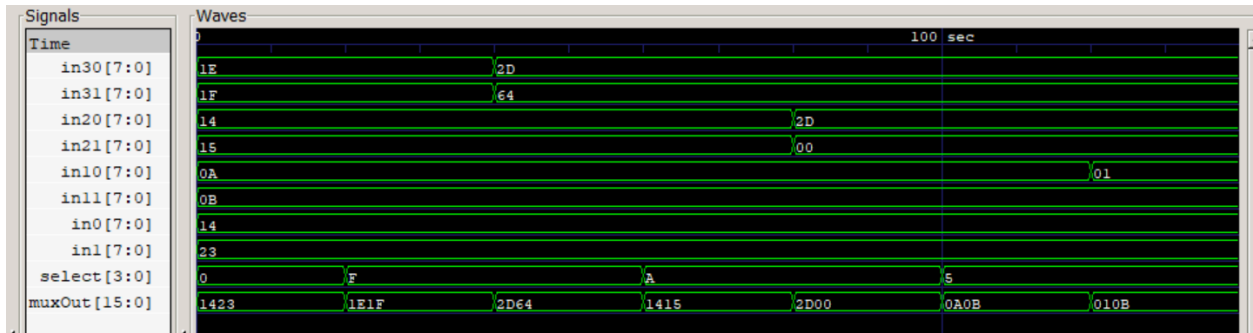
decoder4to16:



register8bit:



mux32to2:



ram:

